

DD: PLM-CFG-04 — Discount Catalog

Developer implementation guide: DD_PLM-CFG-04-Discount_Catalog-DEV_IMPL_GUIDE-v0.6.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_PLM-CFG-04-Discount_Catalog-v0.6.md.

Verdict: ■ Modify (Promo, PromoType, PromoInfo entities exist in TZ — need consolidation into a single Discount catalog with WalletCredit vs PriceRebate distinction, formal lifecycle with automatic activation, geo-scoping) **Wave:** 4 |

Section A: Configuration & Catalogs

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_PLM-CFG-04-Discount_Catalog-v0.6.md
Version	v0.6
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/discounts/{id}
- GET /api/discounts
- GET /api/discounts/applicable
- GET /api/discounts/{id}
- PATCH /api/discounts/{id}
- POST /api/discounts
- POST /api/discounts/{id}/disable
- POST /api/discounts/{id}/enable
- POST /api/discounts/{id}/retire

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-rules
- discount-catalog

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.

Concern	Recommendation
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ Modify (Promo, PromoType, PromoInfo entities exist in TZ — need consolidation into a single Discount catalog with WalletCredit vs PriceRebate distinction, formal lifecycle with automatic activation, geo-scoping)

Wave: 4 | **Section A:** Configuration & Catalogs

A — Target

1. What this process is about

A **Discount** is a configured reduction of what a customer pays, either as a **PriceRebate** (reduces a billable amount — recurring invoice line, installation fee, MACD fee, or any other charge) or as a **WalletCredit** (credits a customer wallet). The Discount catalog stores the definitions; assignment of a discount to a specific customer/package and the actual application at any billable event are operational processes covered by later DDs (assignment by **DIS-OP-01**, application at recurring cycle close by **BIL-01**, application at onboarding/installation by **SIP-03** and **FUL-03**, application at top-up by **BIL-06**, in campaigns by **EM-04**).

This DD covers the **configuration** of Discounts by an admin user: create, read, update, soft-delete, plus an INACTIVE / ACTIVE / RETIRED lifecycle (with automatic ACTIVE/INACTIVE flipping driven by the validity window) and audit. There is no business workflow here — this is administrative CRUD on catalog data with rules.

PLM-CFG-04 scope. This DD covers:

- The Discount entity (catalog)
- Discount lifecycle (INACTIVE / ACTIVE / RETIRED) with automatic activation/deactivation based on `startDate/endDate`
- Eligibility filters (per-Package, per-Franchise)
- Validity window (Start Date / End Date)
- Two flavors: **PriceRebate** (any billable charge — recurring invoice line, installation fee, MACD fee, one-off charge) and **WalletCredit** (credits a wallet)
- Type (PERCENTAGE / AMOUNT) and value

PLM-CFG-04 NOT in scope (handled in later DDs):

- Customer-to-discount assignment (the `customer_discount` table and assignment APIs) → **DIS-OP-01**
- Batch assignment via file upload → **DIS-OP-01**
- Application at recurring cycle close (postpaid invoice) → **BIL-01**
- Application at onboarding / installation fees → **SIP-03**, **FUL-03**
- Application at prepaid top-up → **BIL-06**
- Application at MACD events → **FUL-04** and related
- Campaign attachment (Discount linked to a marketing campaign) → **EM-04**
- Cancellation of an assigned discount per customer → **DIS-OP-01**

Downstream modules (billing, cvm, search index) consume Discount catalog data via Kafka events.

Module placement. Discount catalog is **owned by billing**, not plm. The PLM family in the code organization holds **product catalog** (Service, Package, Bundle — what the customer buys); discounts are a **revenue/billing concept** alongside Tax, Wallet, Charge type, Penalty type, and Adjustment type. The "PLM-CFG-XX" ID prefix is historical (inherited from the

workshop classification); the module owner is billing.

2. Business rules

ID	Rule	Triggered on
R-PLM-CFG-04-1	Discount code must be unique within the discount catalog.	Create, Update
R-PLM-CFG-04-2	Discount code is immutable once the Discount has been ACTIVE at least once. Code is referenced by ledger transactions (DIS rows on customer ledger, charge-time application records); it must be stable for audit and reporting.	Update
R-PLM-CFG-04-3	Discount flavor is one of PRICE_REBATE or WALLET_CREDIT. PRICE_REBATE reduces any billable amount (recurring invoice line, installation fee, MACD fee, one-off charge). WALLET_CREDIT credits a wallet (the creditWalletRef). flavor is immutable after creation.	Create, Update
R-PLM-CFG-04-4	Discount type is one of PERCENTAGE or AMOUNT. (The third historical type REMAINING_AMOUNT is NOT supported in v1 — never used in KE.)	Create, Update
R-PLM-CFG-04-5	If type = PERCENTAGE, value must be a number in the range (0, 100]. If type = AMOUNT, value must be a positive number expressed in the deployment currency (KES, UGX, TZS, XOF).	Create, Update
R-PLM-CFG-04-6	Discount validity window : startDate ≤ endDate. endDate may be null (open-ended, e.g., staff-rate discount). An endDate of 9999-12-31 is also acceptable for "permanent until cancelled" discounts.	Create, Update
R-PLM-CFG-04-7	A Discount's eligiblePackageRefs (cross-module references to Packages from SIP-01) must all exist and be ACTIVE at the time of Discount enablement (the admin toggles adminEnabled true). The Discount catalog validates by calling plm/packages/{ref} REST and consulting its local Package cache.	Enable (adminEnabled → true)
R-PLM-CFG-04-8	A Discount's eligibleFranchiseRefs (cross-module references to Franchises from HomePass-CFG) must all exist at the time of Discount enablement. Empty eligibleFranchiseRefs means "all franchises".	Enable (adminEnabled → true)
R-PLM-CFG-04-9	A Discount cannot be RETIRED while it has at least one active customer assignment (cross-module check against the customer_discount table owned by DIS-OP-01). To stop a Discount from applying, the admin should either set endDate to a past date (effectiveStatus auto-flips to INACTIVE at the next refresh) or unassign customers first via DIS-OP-01.	Retire
R-PLM-CFG-04-10	A Discount cannot be soft-deleted while it has at least one active customer assignment. Soft-delete is only allowed when effectiveStatus = RETIRED AND activeAssignmentCount = 0.	Delete
R-PLM-CFG-04-11	A WalletCredit Discount must reference a creditWalletRef (which wallet receives the credit), validated against the Wallet catalog (PLM-CFG-03). For PriceRebate, creditWalletRef must be null.	Create, Update
R-PLM-CFG-04-12	A Discount optionally declares eligibleBillableEventRefs[] — an array of BillableEvent codes from BIL-CFG-01 that the discount can apply to. Empty/null means "applies to any billable amount" (the legacy behaviour). When populated, BIL-01 only applies this discount when the charge being processed references a BillableEvent code in the list. Allows fine-grained targeting (e.g., a "20% off installation" discount that does NOT apply to ETF or pause fees). The BillableEvent refs are validated against BIL-CFG-01 at enable time and must reference BillableEvents that are not RETIRED. Cross-module check at enable.	Create, Update, Enable

Lifecycle semantics (no rule ID — these are state machine invariants):

- The Discount has two underlying stored fields and one derived field:
- `adminEnabled` : `boolean` — admin's intent ("I want this Discount to be applicable when in date window"). Default `false` on create.
- `retiredAt` : `timestamp NULL` — set when admin manually retires the Discount. Terminal.
- `effectiveStatus` : `enum {INACTIVE, ACTIVE, RETIRED}` — derived (or cached and refreshed by a daily job; see Sequence diagrams):
- `RETIRED` if `retiredAt IS NOT NULL`
- `ACTIVE` if `retiredAt IS NULL AND adminEnabled = true AND startDate ≤ today ≤ endDate` (with `endDate = null` treated as $+\infty$)
- `INACTIVE` otherwise
- **Initial state on creation:** `effectiveStatus = INACTIVE` (because `adminEnabled` defaults to `false`).
- **Manual transitions** (admin actions):
- Toggle `adminEnabled` (`true ↔ false`) — recomputes `effectiveStatus`. Triggers R-PLM-CFG-04-7 and -8 when going to `true`.
- Retire (`retiredAt = now`) — terminal. Triggers R-PLM-CFG-04-9. Once retired, cannot return to `ACTIVE` or `INACTIVE`.
- **Automatic transitions** (driven by time, not by admin action):
- When `startDate` is reached and `adminEnabled = true`: `effectiveStatus` becomes `ACTIVE` automatically (no admin action needed).
- When `endDate` passes: `effectiveStatus` becomes `INACTIVE` automatically. The Discount is NOT retired — it can be reactivated by extending `endDate`.

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	billing	Owns Discount catalog data, REST APIs, the discount admin UI, the DRL rules KJAR (billing-rules), and the daily <code>effectiveStatus</code> refresh job
Impacted (cross-references)	p1m (Package catalog SIP-01), homepass (Franchise catalog HomePass-CFG), billing (Wallet catalog PLM-CFG-03)	Read at enable time via REST for eligibility validation (R-PLM-CFG-04-7, R-PLM-CFG-04-8, R-PLM-CFG-04-11)
Impacted (downstream consumers)	billing (invoice generation BIL-01 — reads PriceRebate definitions at cycle close), onboarding / fulfillment (SIP-03, FUL-03 — apply at install / first payment), fulfillment (FUL-04 — apply at MACD events), billing (BIL-06 — apply WalletCredit at top-up), cvm (campaigns EM-04), search index, BSS Backoffice	Consume DiscountCreated, DiscountUpdated, DiscountEnabledChanged, DiscountEffectiveStatusChanged, DiscountRetired Kafka events; refresh their local caches
Coordination	billing (DIS-OP-01 customer-discount assignment)	R-PLM-CFG-04-9 and -10 — Discount retire / delete blocked if active customer assignment; the assignment table is owned by DIS-OP-01 in the same billing DB
Gateways	—	None used in this process
Engines	Drools KIE	Rule evaluation on every create/update/enable-toggle/retire/delete

No workflow, no Camunda. No external system calls. PLM-CFG-04 is pure catalog admin.

4. Foundation references

DB → FOUNDATION_DB.md · Drools → FOUNDATION_DROOLS.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md · Auth → FOUNDATION_AUTH.md.

Technical design

Module placement

Layer	Where
Frontend	Shared BSS Backoffice UI (Billing section, Discount Catalog sub-area)
Backend	billing module, sub-area discount-catalog/
Rules	Rules Engine (Drools) — billing-rules KJAR. PLM-CFG-04 contributes its own DRL files and Fact classes; the KJAR is shared with other billing-owned configs (Tax PLM-CFG-02, Wallet PLM-CFG-03, Charge type PLM-CFG-06, Penalty type PLM-CFG-07, Adjustment type PLM-CFG-05).
Events	Internal Event Bus (Kafka), topic family sophix.billing.discount.*

Single-tenant: same codebase per country, config + rule packages loaded at startup.

billing module — ecosystem (PLM-CFG-04 view)

```

flowchart LR
    UI[Backoffice UI Discount Catalog] -->|REST| BIL[billing module discount-catalog]
    BIL -->|invoke| RE[Rules Engine]
    BIL -->|publish| EB[Kafka]
    EB --> INV[billing invoicing BIL-01]
    EB --> WAL[billing wallet BIL-06]
    EB --> CVM[cvm cache]
    EB --> SEARCH[search index]
    BIL -.lookup at enable.-> PLM[plm Package catalog SIP-01]
    BIL -.lookup at enable.-> HP[homepass Franchise]
    BIL -.lookup at enable.-> WALCAT[billing Wallet catalog]
    BIL -.coordinates with.-> DISOP[billing DIS-OP-01 customer_discount]

```

The billing module owns multiple sub-areas in the same codebase (invoice generation, wallet, tax, dunning, refund, discount catalog, etc.). PLM-CFG-04 lives in discount-catalog/.

Codebase

PLM-CFG-04 adds to the billing module. Stack stays as chosen for billing (Java/Spring Boot OR Laravel — same contract).

Java/Spring Boot layout

```

modules/billing/
src/main/java/com/sophix/billing/
  invoicing/                ← BIL-01
  wallet/                   ← BIL-06/07 + PLM-CFG-03
  tax/                      ← PLM-CFG-02
  dunning/                  ← BIL-04
  discount-catalog/        ← PLM-CFG-04 (this DD)
    domain/
      Discount.java
      DiscountStatus.java
      DiscountFlavor.java
      DiscountType.java
      DiscountMethod.java
      DiscountDistributionSchema.java
    repository/
      DiscountRepository.java
    service/
      DiscountService.java
      DiscountValidationService.java ← invokes Drools
    web/rest/
      DiscountResource.java
      dto/{CreateDiscountRequest, UpdateDiscountRequest, DiscountResponse}.java
    event/
      DiscountEventPublisher.java
    exception/
      DiscountException.java
  shared/
  messaging/                ← outbox/inbox workers
src/main/resources/
  config/liquibase/changelog/billing/
    017_discount.xml        ← new for PLM-CFG-04
    018_discount_eligibility.xml ← new for PLM-CFG-04

```

Laravel layout (if billing is Laravel)

```

modules/billing/
  app/
    Domain/Invoicing/        ← BIL-01
    Domain/Wallet/           ← BIL-06/07

```

```

Domain/Tax/
Domain/DiscountCatalog/
  Models/
    Discount.php
  Services/
    DiscountService.php
    DiscountValidationService.php
Http/Controllers/
  DiscountController.php
Http/Requests/
  CreateDiscountRequest.php
  UpdateDiscountRequest.php
Events/
  DiscountCreated.php
  DiscountUpdated.php
  DiscountActivated.php
  DiscountRetired.php
Domain/Shared/
Domain/Messaging/
database/migrations/
  2026_05_12_000017_create_discount_table.php
  2026_05_12_000018_create_discount_eligibility_tables.php

```

← PLM-CFG-02
← PLM-CFG-04 (this DD)
← HTTP call to KIE Server
← outbox/inbox workers

In both stacks, DRL files live in the separate Java/Maven KJAR project modules/billing-rules/ (see FOUNDATION_DR00LS.md). Laravel billing calls KIE Server over HTTPS REST.

Foundation-specific configurations

Foundation	Configuration for PLM-CFG-04
DB	billing database in cluster B (Capture & Ops). New tables discount, discount_eligible_package, discount_eligible_franchise in schema discount_catalog. Messaging tables in shared messaging schema.
Drools	KJAR artifact: billing-rules-{country}_{version}.jar (shared with other billing configs). PLM-CFG-04 contributes DRL files under rules/discount-catalog/ and Fact classes under com.sophix.billing.facts.discount.*.
Kafka	Topic family sophix.billing.discount.*. Events: DiscountCreated, DiscountUpdated, DiscountActivated, DiscountRetired. Key format: billing:discount:{discountId}.
Cache	Redis cache for Discount catalog reads by other consumers: sophix:billing:discount:{discountId} with 24h TTL. Invalidated on Kafka events.
Auth	Roles: BILLING_ADMIN (full CRUD + enable/disable + retire), BILLING_VIEWER (read-only), BILLING_ASSIGNER (read catalog + assign via DIS-OP-01 — but assignment is not in this DD). JWT verified locally by billing.

Frontend

BSS Backoffice UI, Billing section, Discount Catalog sub-area. Four screens.

Screen	Purpose
Discount list	Search, filter (by flavor, effectiveStatus, adminEnabled, validity window), paginate. Columns: code, name, flavor, type, value, effectiveStatus badge, adminEnabled toggle indicator, startDate, endDate.
Discount detail (view)	All fields read-only + Eligible Packages + Eligible Franchises + audit log. Shows both adminEnabled and the derived effectiveStatus.
Discount create / edit	Form with all fields; eligibility selectors (multi-pick Packages, multi-pick Franchises); rules feedback inline on save. Fields are editable per the "Editable in" column below — most are editable any time before retire; only code and flavor lock once the Discount has been ACTIVE at least once.
Discount lifecycle actions	Enable / Disable (toggle adminEnabled; on enable, runs eligibility validation R-7 + R-8); Retire (retiredAt = now, terminal, runs R-9); Delete (soft-delete, runs R-10).

Form fields (Discount create / edit):

Field	Type	Editable in	Validation	
code	text (max 50)	before first ACTIVE only	unique; uppercase + digits + underscore	

Field	Type	Editable in	Validation	
name	text (max 200)	any state pre-retire	required	
description	textarea	any state pre-retire	optional	
flavor	radio	before first ACTIVE only	PRICE_REBATE \	WALLET_CREDIT
type	radio	any state pre-retire	PERCENTAGE \	AMOUNT
value	numeric	any state pre-retire	range per rule R-PLM-CFG-04-5	
startDate	date	any state pre-retire	per rule R-PLM-CFG-04-6	
endDate	date	any state pre-retire	per rule R-PLM-CFG-04-6	
creditWalletRef	dropdown (Wallets)	before first ACTIVE only	required if flavor=WALLET_CREDIT	
eligiblePackageRefs[]	multi-select	any state pre-retire	each must exist; validated when admin enables	
eligibleFranchiseRefs[]	multi-select	any state pre-retire	each must exist; validated when admin enables; empty = all franchises	
adminEnabled	toggle	any state pre-retire	manual toggle; on true → rules R-7 + R-8 run	

Status badges:

- INACTIVE (grey) — admin disabled OR outside validity window
- ACTIVE (green) — admin enabled AND inside validity window; this Discount currently applies to new billable events
- RETIRED (red, struck-through) — terminal; cannot be assigned or applied

The badge shows effectiveStatus; a small "■ Enabled" / "■ Disabled" indicator shows adminEnabled separately so admins see both intent and current effect at a glance.

Data model

discount

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	rule R-PLM-CFG-04-1		
name	TEXT NOT NULL			
description	TEXT NULL			
flavor	TEXT NOT NULL	PRICE_REBATE \	WALLET_CREDIT	
type	TEXT NOT NULL	PERCENTAGE \	AMOUNT	
value	NUMERIC(12,4) NOT NULL			
start_date	DATE NOT NULL			
end_date	DATE NULL			
credit_wallet_ref	TEXT NULL	FK reference to PLM-CFG-03 Wallet; required if flavor=WALLET_CREDIT		
admin_enabled	BOOLEAN NOT NULL DEFAULT false	admin's intent (R-7 + R-8 fire when this flips to true)		
effective_status	TEXT NOT NULL	derived from admin_enabled + start_date + end_date + retired_at; computed and refreshed by daily job (and synchronously updated on any field change touching the inputs). Values: INACTIVE \	ACTIVE \	RETIRED
effective_status_refreshed_at	TIMESTAMP NOT NULL	last time effective_status was recomputed		

Column	Type	Notes		
has_been_active	BOOLEAN NOT NULL DEFAULT false	latches true the first time effective_status becomes ACTIVE; used by R-2 (code immutability) and field-edit rules		
retired_at	TIMESTAMP NULL	non-null = terminal RETIRED		
created_at	TIMESTAMP NOT NULL			
created_by	TEXT NOT NULL			
updated_at	TIMESTAMP NOT NULL			
updated_by	TEXT NOT NULL			
deleted_at	TIMESTAMP NULL	soft-delete; only allowed when retired_at IS NOT NULL AND no active assignments		

Indexes:

- idx_discount_code (UNIQUE)
- idx_discount_eff_status_validity (effective_status, start_date, end_date) — supports BIL-01 / SIP-03 / FUL-03 lookup of currently-applicable discounts
- idx_discount_refresh_due partial on (end_date) WHERE effective_status = 'ACTIVE' — supports the daily refresh job efficiently finding ACTIVE Discounts whose end_date has passed

discount_eligible_package

Column	Type	Notes
discount_id	TEXT NOT NULL	FK discount.id
package_ref	TEXT NOT NULL	cross-module ref to SIP-01 Package (no FK across modules)
created_at	TIMESTAMP NOT NULL	

Primary key: (discount_id, package_ref). **Index:** idx_discount_pkg_lookup on (package_ref) — supports reverse lookup ("which discounts apply to package X").

discount_eligible_franchise

Column	Type	Notes
discount_id	TEXT NOT NULL	FK discount.id
franchise_ref	TEXT NOT NULL	cross-module ref to HomePass-CFG Franchise
created_at	TIMESTAMP NOT NULL	

Primary key: (discount_id, franchise_ref). **Index:** idx_discount_fran_lookup on (franchise_ref).

Note: empty discount_eligible_franchise for a Discount means "all franchises" (per R-PLM-CFG-04-10).

discount_eligible_billable_event

Per R-PLM-CFG-04-12 — targets specific BillableEvents from BIL-CFG-01.

Column	Type	Notes
discount_id	TEXT NOT NULL	FK discount.id
billable_event_ref	TEXT NOT NULL	cross-module ref to BIL-CFG-01 BillableEvent code (no FK across modules)
created_at	TIMESTAMP NOT NULL	

Primary key: (discount_id, billable_event_ref). **Index:** idx_discount_be_lookup on (billable_event_ref) — supports reverse lookup ("which discounts apply to BillableEvent X") at charge time.

Note: empty discount_eligible_billable_event for a Discount means "applies to any billable amount" (legacy behavior, per R-PLM-CFG-04-12).

audit_log_entry (shared by all billing features)

Standard audit pattern (per HLD §13). Each operation on a Discount logs to `billing.audit_log_entry` with `entity_type='Discount'`, `entity_id=discount.id`, `action`, `actor`, `changed_fields_json`, `created_at`.

Messaging tables

Live in billing DB, schema messaging (per FOUNDATION_KAFKA.md):

- `messaging.outbox_event` — events emitted by PLM-CFG-04
- `messaging.inbox_event` — events consumed (none in this DD; PLM-CFG-04 is read-only for cross-module data)

Outbox worker pseudocode

```
// runs every 500ms
List<OutboxEvent> pending = repo.findPending(100); // partial index
for (OutboxEvent e : pending) {
    try {
        kafkaProducer.send(e.topic(), e.key(), e.envelope())
            .get(5, SECONDS); // acks=all
        repo.markPublished(e.id(), now());
    } catch (Exception ex) {
        repo.incrementAttempts(e.id(), ex.getMessage());
    }
}
```

Cleanup

Table	Retention
<code>messaging.outbox_event</code>	30 days after <code>published_at</code>
<code>messaging.inbox_event</code>	7 days after <code>processed_at</code>
<code>audit_log_entry</code> (Discount)	indefinite (compliance)

Sequence diagrams

Create Discount (initial state INACTIVE)

```
sequenceDiagram
    participant U as Billing Admin
    participant UI as Backoffice UI
    participant BIL as billing module
    participant KIE as KIE Server
    participant DB as billing DB
    participant K as Kafka

    U->>UI: fill create form with adminEnabled false
    UI->>BIL: POST discounts with JWT
    BIL->>KIE: evaluate create rules
    KIE-->>BIL: ok
    BIL->>BIL: compute effective_status as INACTIVE
    BIL->>DB: INSERT discount and eligibility rows
    BIL->>DB: INSERT outbox DiscountCreated
    BIL->>K: outbox worker publishes DiscountCreated
    BIL-->>UI: 201 Created
```

Enable Discount (toggle adminEnabled → true)

```
sequenceDiagram
    participant U as Billing Admin
    participant BIL as billing module
    participant KIE as KIE Server
    participant PLM as plm module
    participant HP as homepass module
    participant DB as billing DB
    participant K as Kafka

    U->>BIL: POST enable discount
    BIL->>KIE: evaluate enable rules R-7 and R-8
    KIE-->>BIL: needs cross-module verification
    BIL->>PLM: GET packages by refs verify ACTIVE
    PLM-->>BIL: 200 ok or 404 missing
    BIL->>HP: GET franchises by refs
    HP-->>BIL: 200 ok or 404 missing
    BIL->>BIL: compute effective_status
    BIL->>DB: UPDATE discount admin_enabled and effective_status
    BIL->>DB: INSERT outbox DiscountEnabledChanged
    BIL->>K: events published
    BIL-->>U: 200 OK
```

Disable Discount (toggle adminEnabled → false)

```
sequenceDiagram
    participant U as Billing Admin
    participant BIL as billing module
    participant DB as billing DB
    participant K as Kafka

    U->>BIL: POST disable discount
    BIL->>BIL: effective_status recomputes to INACTIVE
    BIL->>DB: UPDATE discount admin_enabled false
    BIL->>DB: INSERT outbox DiscountEnabledChanged
    BIL->>K: events published
    BIL-->>U: 200 OK
```

Note: disable does NOT need eligibility rules. Customer assignments remain in place; they're idle while the Discount is INACTIVE. Toggling back to enabled re-runs R-7 + R-8.

Daily effectiveStatus refresh job

```
sequenceDiagram
    participant CRON as Daily scheduler
    participant BIL as billing module
    participant DB as billing DB
    participant K as Kafka

    CRON->>BIL: trigger refresh just after midnight
    BIL->>DB: SELECT discounts whose effective_status would change
    DB-->>BIL: list of discounts
    loop for each
        BIL->>BIL: compute new effective_status
        BIL->>DB: UPDATE discount effective_status
        BIL->>DB: INSERT outbox DiscountEffectiveStatusChanged
    end
    BIL-->>K: outbox worker publishes events
```

The job is small (only flips at date boundaries). For real-time accuracy between job runs, callers that need current applicability (BIL-01 at cycle close, SIP-03 at onboarding) recompute effectiveStatus inline using the start_date/end_date/admin_enabled/retired_at fields rather than trusting the cached value alone. The cached value drives indexed queries; inline recomputation is the source of truth at decision time.

Retire Discount (terminal)

```
sequenceDiagram
    participant U as Billing Admin
    participant BIL as billing module
    participant DISOP as billing module DIS-OP-01
    participant KIE as KIE Server
    participant DB as billing DB
    participant K as Kafka

    U->>BIL: POST retire discount
    BIL->>DISOP: SELECT active customer_discount count
    DISOP-->>BIL: count
    BIL->>KIE: evaluate R-9 count must be zero
    KIE-->>BIL: ok or blocked
    alt count is zero
        BIL->>DB: UPDATE discount retired_at
        BIL->>DB: INSERT outbox DiscountRetired
        BIL->>K: DiscountRetired event
        BIL-->>U: 200 OK
    else count above zero
        BIL-->>U: 409 Conflict with active count
    end
```

Soft-delete Discount (only when RETIRED and no assignments)

```
sequenceDiagram
    participant U as Billing Admin
    participant BIL as billing module
    participant DISOP as billing module DIS-OP-01
    participant KIE as KIE Server
    participant DB as billing DB
    participant K as Kafka

    U->>BIL: DELETE discount
    BIL->>DISOP: SELECT active customer_discount count
    DISOP-->>BIL: count
    BIL->>KIE: evaluate R-10 must be RETIRED and count zero
    KIE-->>BIL: ok or blocked
    alt allowed
        BIL->>DB: UPDATE discount deleted_at
        BIL->>DB: INSERT outbox DiscountUpdated
        BIL->>K: DiscountUpdated event
    end
```

```

        BIL-->U: 200 OK
    else blocked
        BIL-->U: 409 Conflict
    end

```

Rules (Drools)

PLM-CFG-04 adds its DRL files and Fact classes to the existing billing-rules KJAR.

DRL files

```

modules/billing-rules/
src/main/resources/
  META-INF/
    kmodule.xml                ← updated to include new DRLs
  rules/
    tax/                       ← PLM-CFG-02
    wallet/                    ← PLM-CFG-03
    discount-catalog/
      discount-validation-create-update.drl ← new
      discount-validation-enable.drl       ← new
      discount-validation-retire.drl       ← new
      discount-validation-delete.drl       ← new

```

No discount-defaults.drl is needed — there are no defaultable fields in v1.

kmodule.xml (additive)

```

<kmodule xmlns="http://www.drools.org/xsd/kmodule">
  <kbase name="billing" packages="rules.tax, rules.wallet, rules.discount-catalog">
    <ksession name="billingSession" type="stateless"/>
  </kbase>
</kmodule>

```

Fact classes (Java, Lombok)

```

package com.sophix.billing.facts.discount;

import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class DiscountFact {
    private String id;
    private String code;
    private String name;
    private String description;
    private String flavor;                // PRICE_REBATE | WALLET_CREDIT
    private String type;                  // PERCENTAGE | AMOUNT
    private BigDecimal value;
    private LocalDate startDate;
    private LocalDate endDate;
    private String creditWalletRef;        // nullable; required if flavor=WALLET_CREDIT
    private boolean adminEnabled;
    private String effectiveStatus;        // INACTIVE | ACTIVE | RETIRED
    private boolean hasBeenActive;        // for R-2 (code immutability)
    private List<String> eligiblePackageRefs;
    private List<String> eligibleFranchiseRefs;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class DiscountCodeUniquenessFact {
    private String code;
    private boolean exists;              // populated by service before invoking rules
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class PackageRefFact {
    private String packageRef;
    private boolean exists;
    private boolean active;
}

```

```

@Data
@NoArgsConstructor
@AllArgsConstructor
public class FranchiseRefFact {
    private String franchiseRef;
    private boolean exists;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletRefFact {
    private String walletRef;
    private boolean exists;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class CustomerDiscountAssignmentCountFact {
    private String discountId;
    private int activeAssignmentCount;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class DiscountValidationOutcome {
    private boolean valid;
    private List<String> errors;
    private List<String> warnings;
}

```

discount-validation-create-update.drl

```

package rules.discount-catalog;

import com.sophix.billing.facts.discount.DiscountFact;
import com.sophix.billing.facts.discount.DiscountCodeUniquenessFact;
import com.sophix.billing.facts.discount.WalletRefFact;
import com.sophix.billing.facts.discount.DiscountValidationOutcome;

rule "R-PLM-CFG-04-1: code must be unique"
    when
        $d : DiscountFact()
        $u : DiscountCodeUniquenessFact(code == $d.code, exists == true)
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-1: Discount code '" + $d.getCode() + "' already exists");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-3: flavor must be PRICE_REBATE or WALLET_CREDIT"
    when
        $d : DiscountFact(flavor != "PRICE_REBATE" && flavor != "WALLET_CREDIT")
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-3: flavor must be PRICE_REBATE or WALLET_CREDIT (got '" + $d.getFlavor() + "')");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-4: type must be PERCENTAGE or AMOUNT"
    when
        $d : DiscountFact(type != "PERCENTAGE" && type != "AMOUNT")
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-4: type must be PERCENTAGE or AMOUNT (got '" + $d.getType() + "')");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-5a: PERCENTAGE value in (0, 100]"
    when
        $d : DiscountFact(type == "PERCENTAGE", value != null, value.compareTo(java.math.BigDecimal.ZERO) <= 0 || value.co
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-5: PERCENTAGE value must be in (0, 100], got " + $d.getValue());
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-5b: AMOUNT value positive"
    when
        $d : DiscountFact(type == "AMOUNT", value != null, value.compareTo(java.math.BigDecimal.ZERO) <= 0)
        $out : DiscountValidationOutcome()

```

```

        then
            $out.getErrors().add("R-PLM-CFG-04-5: AMOUNT value must be positive, got " + $d.getValue());
            modify($out) { setValid(false); }
        end

rule "R-PLM-CFG-04-6: startDate must be ≤ endDate when endDate is not null"
    when
        $d : DiscountFact(startDate != null, endDate != null, startDate.isAfter(endDate))
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-6: startDate (" + $d.getStartDate() + ") must be ≤ endDate (" + $d.getEndDate());
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-11a: WALLET_CREDIT requires creditWalletRef"
    when
        $d : DiscountFact(flavor == "WALLET_CREDIT", creditWalletRef == null || creditWalletRef == "")
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-11: flavor WALLET_CREDIT requires a creditWalletRef");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-11b: PRICE_REBATE forbids creditWalletRef"
    when
        $d : DiscountFact(flavor == "PRICE_REBATE", creditWalletRef != null && creditWalletRef != "")
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-11: flavor PRICE_REBATE must not have creditWalletRef set");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-11c: creditWalletRef must exist if set"
    when
        $d : DiscountFact(flavor == "WALLET_CREDIT", creditWalletRef != null)
        $w : WalletRefFact(walletRef == $d.creditWalletRef, exists == false)
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-11: creditWalletRef '" + $d.getCreditWalletRef() + "' does not exist in Wallet");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-2a: code immutable once Discount has been ACTIVE"
    when
        $d : DiscountFact(hasBeenActive == true)
        $u : DiscountCodeUniquenessFact(code == $d.code, exists == false) // user is trying to change code to a new one
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-2: Discount code is immutable once the Discount has been ACTIVE");
        modify($out) { setValid(false); }
    end

rule "R-PLM-CFG-04-3b: flavor immutable after creation"
    when
        $d : DiscountFact(hasBeenActive == true, flavor != null) // service layer compares against stored flavor before in
        $out : DiscountValidationOutcome()
        eval(false) // placeholder; service layer enforces flavor immutability before invoking rules
    then
        // not used; immutability checked at service layer prior to rules invocation
    end

```

Note: R-2 (code immutability) and R-3 flavor immutability are partially enforced in the service layer before invoking rules — the rule engine sees the **attempted new state** and the **current stored state** is checked Java-side. This is a pattern from PLM-CFG-01: not every constraint needs to be in DRL; immutability of identifiers is structural and lives in service code.

discount-validation-enable.drl

```

package rules.discount-catalog;

import com.sophix.billing.facts.discount.DiscountFact;
import com.sophix.billing.facts.discount.PackageRefFact;
import com.sophix.billing.facts.discount.FranchiseRefFact;
import com.sophix.billing.facts.discount.DiscountValidationOutcome;

rule "R-PLM-CFG-04-7: eligible package must exist and be ACTIVE at enable"
    when
        $d : DiscountFact()
        $p : PackageRefFact(packageRef memberOf $d.eligiblePackageRefs, (exists == false || active == false))
        $out : DiscountValidationOutcome()
    then
        String msg = "R-PLM-CFG-04-7: eligible Package '" + $p.getPackageRef() + "' ";
        msg += (!$p.isExists()) ? "does not exist" : "is not ACTIVE";
    end

```

```

        $out.getErrors().add(msg);
        modify($out) { setValid(false); }
    end

    rule "R-PLM-CFG-04-8: eligible franchise must exist if specified"
    when
        $d : DiscountFact()
        $f : FranchiseRefFact(franchiseRef memberOf $d.eligibleFranchiseRefs, exists == false)
        $out : DiscountValidationOutcome()
    then
        $out.getErrors().add("R-PLM-CFG-04-8: eligible Franchise '" + $f.getFranchiseRef() + "' does not exist");
        modify($out) { setValid(false); }
    end
end

```

discount-validation-retire.drl

```

package rules.discount-catalog;

import com.sophix.billing.facts.discount.DiscountFact;
import com.sophix.billing.facts.discount.CustomerDiscountAssignmentCountFact;
import com.sophix.billing.facts.discount.DiscountValidationOutcome;

rule "R-PLM-CFG-04-9: cannot retire if active customer assignments exist"
when
    $d : DiscountFact()
    $c : CustomerDiscountAssignmentCountFact(discountId == $d.id, activeAssignmentCount > 0)
    $out : DiscountValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-04-9: Discount '" + $d.getCode() + "' has " + $c.getActiveAssignmentCount() + " a
    modify($out) { setValid(false); }
end

```

discount-validation-delete.drl

```

package rules.discount-catalog;

import com.sophix.billing.facts.discount.DiscountFact;
import com.sophix.billing.facts.discount.CustomerDiscountAssignmentCountFact;
import com.sophix.billing.facts.discount.DiscountValidationOutcome;

rule "R-PLM-CFG-04-10a: can only soft-delete if RETIRED"
when
    $d : DiscountFact(effectiveStatus != "RETIRED")
    $out : DiscountValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-04-10: Discount can only be soft-deleted from RETIRED state (current: " + $d.getEf
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-04-10b: cannot soft-delete if active customer assignments exist"
when
    $d : DiscountFact()
    $c : CustomerDiscountAssignmentCountFact(discountId == $d.id, activeAssignmentCount > 0)
    $out : DiscountValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-04-10: Discount '" + $d.getCode() + "' has " + $c.getActiveAssignmentCount() + " a
    modify($out) { setValid(false); }
end

```

Build/deploy KJAR

KJAR build and deploy mechanics are identical to PLM-CFG-01 (see DD_PLM-CFG-01.md §"Build and deploy KJAR" for the full reference). Brief recap:

- Maven build per country: billing-rules-ke_1.0.0.jar, billing-rules-ug_1.0.0.jar, etc.
- Published to internal Maven repository on every change
- KIE Server (2 nodes, behind NGINX) pulls latest version per country at startup
- Hot-reload on new version available (Drools KIE Scanner)

Events (Kafka)

All events on topic family `sophix.billing.discount.*`. Key format: `billing:discount:{discountId}`.

sophix.billing.discount.created

```

{
    "eventId": "billing:discount:dsc_01HX:evt_01JZ5K6...",
    "eventType": "DiscountCreated",

```

```

    "aggregateId": "dsc_01HX...",
    "timestamp": "2026-05-12T10:23:00Z",
    "actor": "billing.admin@wananchi.co.ke",
    "correlationId": "corr_01HX...",
    "payload": {
      "discountId": "dsc_01HX...",
      "code": "STAFF_HSI_30",
      "name": "Staff HSI 30% discount",
      "flavor": "PRICE_REBATE",
      "type": "PERCENTAGE",
      "value": 30,
      "startDate": "2026-06-01",
      "endDate": "9999-12-31",
      "creditWalletRef": null,
      "adminEnabled": false,
      "effectiveStatus": "INACTIVE",
      "eligiblePackageRefs": ["pkg_01HX...", "pkg_02AB..."],
      "eligibleFranchiseRefs": []
    }
  }
}

```

sophix.billing.discount.updated

Same envelope shape; pay load contains the full discount state after update. Emitted on field updates (including adminEnabled toggles — see below) and on soft-delete (in which case pay load.deletedAt is populated).

sophix.billing.discount.enabledchanged

Emitted when the admin toggles adminEnabled. Carries both old and new values so consumers can react.

```

{
  "eventId": "billing:discount:dsc_01HX:evt_01JZ5K9...",
  "eventType": "DiscountEnabledChanged",
  "aggregateId": "dsc_01HX...",
  "timestamp": "2026-05-12T11:00:00Z",
  "actor": "billing.admin@wananchi.co.ke",
  "correlationId": "corr_01HX...",
  "payload": {
    "discountId": "dsc_01HX...",
    "code": "STAFF_HSI_30",
    "previousAdminEnabled": false,
    "newAdminEnabled": true
  }
}

```

sophix.billing.discount.effectivestatuschanged

Emitted when effectiveStatus flips. This is the event downstream consumers (BIL-01, SIP-03, FUL-03, BIL-06) primarily care about — it says "this Discount now applies / no longer applies to new billable events." Emitted both on admin actions and on automatic date-driven flips by the daily refresh job.

```

{
  "eventId": "billing:discount:dsc_01HX:evt_01JZ5KA...",
  "eventType": "DiscountEffectiveStatusChanged",
  "aggregateId": "dsc_01HX...",
  "timestamp": "2026-06-01T00:05:00Z",
  "actor": "system:daily-refresh",
  "correlationId": "job_01JZ5KA...",
  "payload": {
    "discountId": "dsc_01HX...",
    "code": "STAFF_HSI_30",
    "previousEffectiveStatus": "INACTIVE",
    "newEffectiveStatus": "ACTIVE",
    "reason": "startDate reached"
  }
}

```

reason values: admin enabled, admin disabled, startDate reached, endDate passed, retired.

sophix.billing.discount.retired

```

{
  "eventId": "billing:discount:dsc_01HX:evt_01JZ5KB...",
  "eventType": "DiscountRetired",
  "aggregateId": "dsc_01HX...",
  "timestamp": "2026-05-12T12:00:00Z",
  "actor": "billing.admin@wananchi.co.ke",
  "correlationId": "corr_01HX...",
  "payload": {
    "discountId": "dsc_01HX...",
    "code": "STAFF_HSI_30",
    "retiredAt": "2026-05-12T12:00:00Z"
  }
}

```



```
}  
}
```

Exceptions

Standard taxonomy (per HLD). HTTP mapping in the APIs section below.

Exception	When	HTTP
DiscountValidationException	Drools rules outcome has errors	422
DiscountNotFoundException	GET / lifecycle action on unknown id	404
DiscountConflictException	Operation conflicts with current state (e.g., enable while RETIRED, delete while assignments exist)	409
CrossModuleRefException	Package / Franchise / Wallet ref invalid	422
AuthorizationException	Caller missing required role	403

APIs

All endpoints under `/api/discounts`. Auth: Bearer JWT, role per endpoint.

Status codes

Code	Meaning
200	OK (read, list, enable, disable, retire, delete)
201	Created
204	No Content (only used internally; not for human callers in this DD)
400	Bad Request (malformed JSON, wrong types)
401	Unauthorized (no/invalid JWT)
403	Forbidden (role missing)
404	Not Found
409	Conflict (state-machine violation, e.g., enabling a RETIRED Discount)
422	Unprocessable Entity (rules failed)
500	Server error (logged; client should retry)

GET `/api/discounts` — list

Roles: BILLING_ADMIN, BILLING_VIEWER, BILLING_ASSIGNER.

Query parameters:

- `flavor` (optional): PRICE_REBATE | WALLET_CREDIT
- `effectiveStatus` (optional): INACTIVE | ACTIVE | RETIRED
- `adminEnabled` (optional boolean): filter by admin intent
- `validOn` (optional, ISO date): only Discounts with `startDate ≤ validOn ≤ endDate`
- `page`, `size`: pagination

Response 200:

```
{  
  "items": [  
    {  
      "id": "dsc_01HX...",  
      "code": "STAFF_HSI_30",  
      "name": "Staff HSI 30% discount",  
      "flavor": "PRICE_REBATE",  
      "type": "PERCENTAGE",  
      "value": 30,  
      "adminEnabled": true,  
      "effectiveStatus": "ACTIVE",  
      "startDate": "2026-06-01",  
      "endDate": "9999-12-31"  
    }  
  ],  
  "page": 0,  
  "size": 50,  
  "total": 1  
}
```

```
}
```

GET /api/discounts/{id} — read one

Roles: BILLING_ADMIN, BILLING_VIEWER, BILLING_ASSIGNER.

Response 200: full DiscountResponse including eligiblePackageRefs[] and eligibleFranchiseRefs[].

```
{
  "id": "dsc_01HX...",
  "code": "STAFF_HSI_30",
  "name": "Staff HSI 30% discount",
  "description": "Permanent 30% off internet for staff",
  "flavor": "PRICE_REBATE",
  "type": "PERCENTAGE",
  "value": 30,
  "startDate": "2026-06-01",
  "endDate": "9999-12-31",
  "creditWalletRef": null,
  "adminEnabled": true,
  "effectiveStatus": "ACTIVE",
  "effectiveStatusRefreshedAt": "2026-05-12T00:05:00Z",
  "hasBeenActive": true,
  "retiredAt": null,
  "eligiblePackageRefs": ["pkg_01HX...", "pkg_02AB..."],
  "eligibleFranchiseRefs": [],
  "createdAt": "2026-05-12T10:23:00Z",
  "createdBy": "billing.admin@wananchi.co.ke",
  "updatedAt": "2026-05-12T11:00:00Z",
  "updatedBy": "billing.admin@wananchi.co.ke"
}
```

POST /api/discounts — create (INACTIVE)

Roles: BILLING_ADMIN.

Request:

```
{
  "code": "STAFF_HSI_30",
  "name": "Staff HSI 30% discount",
  "description": "Permanent 30% off internet for staff",
  "flavor": "PRICE_REBATE",
  "type": "PERCENTAGE",
  "value": 30,
  "startDate": "2026-06-01",
  "endDate": null,
  "creditWalletRef": null,
  "eligiblePackageRefs": ["pkg_01HX...", "pkg_02AB..."],
  "eligibleFranchiseRefs": []
}
```

Response 201: full DiscountResponse with effectiveStatus = "INACTIVE", adminEnabled = false. Response 422: rules failed.

```
{
  "errors": [
    "R-PLM-CFG-04-1: Discount code 'STAFF_HSI_30' already exists"
  ]
}
```

PATCH /api/discounts/{id} — update

Roles: BILLING_ADMIN.

Editable fields depend on whether the Discount has been ACTIVE at least once (see Frontend section). Body contains only fields to update.

Response 200 / 422 / 404 / 409.

POST /api/discounts/{id}/enable — set adminEnabled = true

Roles: BILLING_ADMIN.

Runs enable rules (R-7, R-8) including cross-module REST verification (Package + Franchise). No body. Recomputes effectiveStatus: if today is in [startDate, endDate], becomes ACTIVE; otherwise stays INACTIVE pending the date window.

Response 200: returns the updated DiscountResponse. Response 422: cross-module ref failures. Response 409: Discount is RETIRED.

POST /api/discounts/{id}/disable — set adminEnabled = false

Roles: BILLING_ADMIN.

No rules to check. Sets adminEnabled = false; effectiveStatus recomputes to INACTIVE. No body.

Response 200 / 409 (Discount is RETIRED).

POST /api/discounts/{id}/retire — terminal

Roles: BILLING_ADMIN.

Runs retire rule (R-9). Sets retiredAt=now, effectiveStatus=RETIRED. No body.

Response 200 / 422 (active assignments exist) / 409 (already retired).

DELETE /api/discounts/{id} — soft-delete

Roles: BILLING_ADMIN.

Runs delete rules (R-10): requires effectiveStatus = RETIRED AND no active assignments. No body.

Response 200 / 422 (not RETIRED, or active assignments) / 404.

GET /api/discounts/applicable — operational lookup (used by BIL-01, SIP-03, FUL-03, BIL-06)

Roles: BILLING_ADMIN, BILLING_INVOICING_SVC (service-to-service).

Query parameters:

- packageRef (required)
- franchiseRef (optional; if omitted, returns all)
- validOn (ISO date; default today)
- flavor (optional): filter by PRICE_REBATE or WALLET_CREDIT

Returns Discounts whose:

- effectiveStatus = ACTIVE (computed inline with validOn against startDate/endDate/adminEnabled/retiredAt — does not trust the cached field if it would be stale for the asked date)
- eligiblePackageRefs contains packageRef
- eligibleFranchiseRefs is empty OR contains franchiseRef

This endpoint is the one BIL-01, SIP-03, FUL-03, BIL-06 call when deciding which Discounts to apply at a billable event.

Response 200: array of DiscountResponse.

B — Current TZ

The TZ Sophix has a partial discount model spread across **Onboarding (Java)** and **Billing (Java)** modules:

TZ entity / class	Stack	Role
Promo	Java billing (sofxBillingModule)	Discount catalog row; fields include code, type, value, startDate, endDate, active flag
PromoType	Java billing	enum-like type registry
PromoInfo	Java billing	per-offer attachment of a Promo
PromoController (in sofxAccount)	Java billing	CRUD endpoints, attach-to-offer, eligibility check, by-subscription lookup, disable
BulkPromoUploadController	Java onboarding	batch upload of customer-to-promo assignments

What TZ already does well:

- Promo lifecycle CRUD with start/end date and active flag
- Eligibility config (attach to specific offers)
- Disable a Promo via PromoController
- Bulk upload of assignments

What's missing for Wananchi KE / Sophix V3:

1. **No flavor distinction.** A Promo in TZ always behaves as a PriceRebate on the bill. Wananchi V3 needs WalletCredit for prepaid customers.
2. **No INACTIVE / ACTIVE / RETIRED lifecycle with automatic date-driven flipping.** TZ has only an active flag — a Promo with active=true and endDate already passed still appears active in queries. V3 separates admin intent (adminEnabled) from current effect (effectiveStatus), and a daily job (plus inline recomputation at decision time) ensures effectiveStatus always reflects today's reality.
3. **No application beyond cycle close.** TZ's PromoController is wired only to the cycle-close apply engine. V3 surfaces the catalog via /api/discounts/applicable so onboarding (SIP-03, FUL-03), upgrades (FUL-04), top-ups (BIL-06) can all apply discounts at the right billable event.
4. **No per-Franchise eligibility.** TZ has per-Offer eligibility (PromoInfo) but no franchise dimension. Wananchi has 6 franchises and assigns staff/loyalty discounts per franchise.
5. **No cross-module ref validation at enable.** TZ doesn't verify Package + Franchise + Wallet refs exist when toggling active; it just toggles. Bad data surfaces later as runtime failures during apply.
6. **No structured DRL rules.** Validation lives inline in Java controllers and service classes; not in a rules engine. Adding new validations requires code change + deploy.
7. **No standard event envelope.** TZ uses internal Java events between the billing and onboarding monoliths; no Kafka publishing for downstream consumers (cache invalidation, search index, CVM, SIP-03, BIL-06) to react to discount changes.
8. BulkPromoUploadController **mixes catalog and assignment.** In V3 these are separated: catalog in PLM-CFG-04 (this DD), assignment in DIS-OP-01.
9. **No soft-delete with assignment-count gate.** Today, an ops user can disable a Promo regardless of in-flight assignments; effect on running customers is ambiguous.

C — Gap & bridge

Migration approach: strangler-fig per discount catalog

The Discount catalog is low-frequency admin data. Strangler-fig is cleanest:

1. **Build the new discount-catalog sub-area** in V3 billing module with the schema, rules, events, APIs from this DD.
2. **Initial seed from TZ:** one-off Liquibase data migration imports existing TZ Promo rows into the new discount table:
 - flavor = PRICE_REBATE for every imported row (TZ has no WalletCredit concept)
 - type mapped 1:1 (PERCENTAGE / AMOUNT)
 - value carried over
 - creditWalletRef = null
 - adminEnabled = (Promo.active = true) (mirror existing intent)
 - effectiveStatus computed at migration time from adminEnabled + startDate + endDate (so ACTIVE Promos in-window come in as ACTIVE; expired ones come in as INACTIVE; manually-deactivated come in as INACTIVE)
 - hasBeenActive = (Promo.active was ever true) (best-effort; can be inferred from PromoInfo attachment history, else default true to be safe — locks code/flavor as if it had been ACTIVE)
 - retiredAt = null initially; admins retire explicitly post-migration if a Promo should be terminal
 - eligiblePackageRefs[] derived from PromoInfo attachments (mapped via Package ref derived from TZ Offer codes)
 - eligibleFranchiseRefs[] empty (all franchises) since TZ has no franchise eligibility
3. **TZ PromoController retired** once all consumers (Wananchi-V3 BIL-01, CVM, SIP-03, etc.) read from the new endpoints.

Dependencies

Dependency	Owner	Status
billing module skeleton with messaging schema, outbox/inbox workers, Drools client	billing team	from foundations
Package catalog (SIP-01) for eligiblePackageRefs ref validation	plm team	■ designed (DD_SIP-01)
Franchise catalog (HomePass-CFG) for eligibleFranchiseRefs ref validation	homepass team	■ pending DD (next in queue after this one)
Wallet catalog (PLM-CFG-03) for creditWalletRef ref validation	billing team	■ pending DD
customer_discount table (DIS-OP-01) for R-PLM-CFG-04-11 active-assignment count check	billing team	■ pending DD
billing-rules KJAR Maven project scaffolding	billing team	created when first billing-rules DD lands (this is it)

What's deferred to other DDs

- **Customer-to-discount assignment** (drag-drop UI, batch upload, schedule-for-future) → **DIS-OP-01**
- **Discount application at recurring cycle close** (compute reduction, distribute across services, post DIS transaction) → **BIL-01** (PriceRebate, postpaid)
- **Discount application at onboarding / installation** (PriceRebate on first payment, installation fee, MACD fee) → **SIP-03, FUL-03, FUL-04**
- **WalletCredit application at top-up** → **BIL-06**
- **Cancel assignment per customer** (supervisor/admin gated, with audit) → **DIS-OP-01**
- **Discount in a Campaign** (Campaign + targeting + multi-customer batch attachment with marketing governance) → **EM-04**
- **Remaining-amount discount type** (never used in KE; deferred indefinitely)
- **Distribution methods/schemas beyond the implicit default** (proportional thru service, original SVC). If a future use case needs a different distribution semantic, add a method field then and re-version the catalog.
- **Auto-renewal of time-bound discounts** (deferred; today every Discount has a single validity window — admins extend endDate manually)
- **Multiple-discount stacking rules** (which discounts compose, which conflict on the same invoice line) — application-time concern; lives in BIL-01 / SIP-03 / etc.

Risks

- **Migration of TZ PromoInfo** → **V3** eligiblePackageRefs: depends on a stable mapping from TZ Offr .code to V3 Package code. To be confirmed against the SIP-01 import.
- **Multiple discounts stacking on the same invoice line**: rules for "which discounts compose, which conflict" are NOT in scope of PLM-CFG-04 (that's an application-time concern, lives in BIL-01). PLM-CFG-04 only ensures each discount is well-formed.
- **Backdating activation**: a Discount activated today with startDate in the past — should it auto-apply retroactively at next cycle close? This is a BIL-01 decision; PLM-CFG-04 allows the configuration but doesn't impose semantics.